Llega el administrador: qué es un SO, cómo arranca y cómo se detiene

Sistemas Operativos · Semana 3

Cap. 2: Introducción a los SO (temas 2.1 – 2.3)

Rubén Rumipamba, Ph.D.

Escuela Politécnica Nacional

Agenda de la semana

SESIÓN 1 · 1 H TEORÍA + 1 H TALLER

¿Quién manda en la cocina?

- Teoría: qué es un SO, por qué nació, U = 1 − p^n, tipos y departamentos
- Taller 1: Conoce a tu administrador (Linux | Windows en parejas)

SESIÓN 2 · 1 H TEORÍA + 1 H TALLER

¿Cómo se organiza y cómo abre el local?

- Teoría: estructura del núcleo, arranque en 5 pasos, parada
- Taller 2: Abre y cierra el local (inicia el Deber 3)

SESIÓN 3 · 1 H · CONSOLIDACIÓN

Resumen, ejercicios y quiz

- Resumen de la semana
- 4 ejercicios con solución
- Quiz final de 5 preguntas
- Deber 3 y revisión autónoma

Objetivos de la semana

RA1 · Explicar	Explicar qué es un sistema operativo, por qué surgió y qué tipos y componentes tiene, con Windows y Linux como ejemplos.
RA1 · Explicar	Describir la secuencia de arranque y parada de Linux y Windows e identificar en qué etapa se detiene un equipo que no arranca.
RA2 · RA3 · Implementar y cooperar	Implementar en pareja un script en Bash y PowerShell que informe cómo arrancó el equipo y subirlo al portafolio de GitHub.

Resultados de aprendizaje de la asignatura: RA1 explicar componentes de la arquitectura y de los SO Windows y Linux · RA2 implementar scripts de administración · RA3 cooperar en equipo.

Sesión 1 · ¿Quién manda en la cocina?

Teoría 55 min (temas 2.1 y 2.2) + Taller 1 «Conoce a tu administrador» 55 min

Llega el administrador: empieza el acto 2

Si hay zonas restringidas, ¿quién manda en la cocina?

1 · La cocina
Semanas 1–2

Hardware y sus límites

2 · Llega el administrador
Semanas 3–6

Qué es el SO, cómo arranca, cómo le hablamos, cómo protege

3 · Muchos a la vez
Semanas 7–11

Procesos, planificación, concurrencia

4 · Los recursos
Semanas 12–16

Memoria, E/S, archivos, respaldos

Abrir el local en la mañana

Arranque y parada · Semana 3

Ventanilla de pedidos

Llamadas al sistema, shell · Semana 4

Proveedores

Paquetes y repositorios · Semana 4

Llaves y permisos

Seguridad · Semana 5

Un edificio con varias cocinas

Virtualización · Semanas 5–6

Estaciones empacadas

Contenedores · Semana 6

La pregunta de la semana

¿Quién es el administrador, qué hace y cómo toma el control de la cocina cada mañana?

¿Por qué hace falta?

Historia breve: del cocinero que mira el horno a la multiprogramación. Sesión 1.

¿Cómo se organiza?

Sus departamentos y su oficina: monolítico, microkernel, módulos. Sesiones 1 y 2.

¿Cómo abre y cierra el local?

Arranque en 5 pasos y parada ordenada. Sesión 2.

¿Qué es un sistema operativo?

El administrador no cocina: organiza

Llega el administrador

No cocina ningún plato: organiza. Y no es otra persona: es un recetario que siguen los mismos cocineros, con el gafete puesto.

Cara 1 · Simplifica

Los cocineros piden «la sal», no «la caja 2 del estante B».

Cara 2 · Reparte

Hay más platos que cocineros: decide a qué plato atiende cada cocinero y da a cada plato su sitio en la mesada y en la bodega.

Y protege

Nadie toma lo de otro plato ni entra sin gafete.

Definición · Un sistema operativo es **software**: el núcleo y los programas de sistema que se ejecutan **en la misma CPU** que las aplicaciones y, con privilegios especiales (modo núcleo), administran el hardware y ofrecen a los programas una forma cómoda y segura de usarlo.

De la cocina a la CPU

En la cocina	En la computadora
El recetario del administrador, seguido con el gafete	**Sistema operativo: software que corre en la CPU**
Pedir «la sal»	**Abstracción: archivo, proceso, socket**
Repartir el tiempo de los cocineros, la mesada y la bodega	**Gestión de recursos: tiempo de CPU (núcleos), RAM, disco**
Gafete y zonas con llave	**Protección: modo núcleo**

Idea clave

Un SO tiene **dos caras**:

máquina extendida: ofrece abstracciones cómodas en lugar de simplemente ofrecer acceso al hardware;

gestor de recursos: reparte el hardware entre muchos programas de forma justa y segura.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.1 · Stallings (2018), Operating Systems: Internals and Design Principles, 9.ª ed., §2.1 · Silberschatz, Galvin y Gagne (2018), Operating System Concepts, 10.ª ed., §1.1

Cara 2 · Gestor de recursos

300 platos para 4 cocineros

```
$ nproc
4
$ top -bn1 | sed -n 2p
Tasks: 312 total,   1 running, 311 sleeping,   0 stopped,   0 zombie
$ vmstat 1 2
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd    free   buff   cache   si   so    bi    bo   in   cs us sy id wa st gu
 1  0      0 4219504   3120 1648220    0    0    41    12  187  293  4  1 95  0  0  0
 0  0      0 4219256   3120 1648232    0    0     0     0  162  241  4  1 95  0  0  0
```

Windows (Administrador de tareas › Rendimiento): Procesos 287 · Subprocesos 3 954 · Núcleos 4

4 cocineros

nproc: 4 núcleos. Solo 4 platos avanzan en el mismo instante.

312 platos

Tasks: 312 procesos en la cocina. ¿Quién decide el turno? El administrador.

311 esperando

sleeping: esperan algo (teclado, red, disco o simplemente un aviso). La CPU está ociosa el **95 % del tiempo** (`id` = 95).

Idea clave

Casi todos los platos están esperando algo. Esa espera es la clave de la historia que sigue.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.1.2 «El SO como gestor de recursos» · Salida de ejemplo, representativa de una VM AlmaLinux 10.2 en reposo (4 núcleos, 8 GiB) · vmstat: la 2.ª línea mide el último segundo

¿Qué es la entrada/salida (E/S)?

Todo lo que un plato necesita de fuera de la cocina

Algo de la bodega

El plato necesita ingredientes guardados, o hay que guardar algo: leer o escribir el **disco**.

Algo del salón

Saber qué eligió el cliente (**teclado, ratón**) o servirle el plato (**pantalla, impresora**).

Algo del proveedor

Lo que no está en el local: la **red**.

El cocinero no va: lo encarga

El dispositivo trabaja solo y tarda; mientras tanto, el plato espera. **Esta semana, todo esto se dibuja como el horno.**



*Para el cocinero, esperar un disco es como esperar **meses**.*

De la cocina a la CPU

En la cocina	En la computadora
Traer o guardar en la bodega	**Leer o escribir el disco (SSD, HDD, USB)**
Lo que pasa en el salón	**Teclado, ratón, pantalla, impresora**
Lo que llega del proveedor	**Red**
El plato está en el horno (convención de la semana)	**El proceso espera una operación de E/S cualquiera**

Idea clave

La E/S es el intercambio de datos entre la CPU y la memoria, por un lado, y los dispositivos, por otro.

Tiene dos rasgos que lo cambian todo: **es muchísimo más lenta que la CPU** y **el dispositivo (no el proceso ni la CPU) trabaja solo**: el proceso espera y la CPU queda libre. Por eso, mientras un plato está en el horno, el cocinero podría avanzar otro… si alguien lo organizara.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.3.5 «Dispositivos de E/S» · Stallings (2018), Operating Systems: Internals and Design Principles, 9.ª ed., cap. 1 · La E/S en detalle (controladores, DMA, búferes): semana 14

Al principio no había administrador

Un cliente a la vez

Un cliente, un plato

El cliente reserva la cocina entera por una hora y la prepara él mismo, a mano.

El plato entra al horno

El horno trabaja solo, pero tarda 20 minutos.

El cocinero mira el horno

Nadie más puede usar la cocina mientras tanto.

Cocina ociosa

Un equipo carísimo esperando sin hacer nada.

De la cocina a la CPU

En la cocina	En la computadora
Cliente que reserva y prepara la cocina	**El programador reserva la máquina y la opera él mismo (carga tarjetas, monta cintas)**
El plato está en el horno	**El proceso espera una E/S (cinta, tarjetas, impresora)**
Cocinero mirando el horno	**CPU ociosa**
Hoja de reservas en la puerta	**Planificación manual (años 50)**

Idea clave

Sin administrador, la CPU pasaba la mayor parte del tiempo **esperando la E/S**: el cocinero miraba el horno sin poder avanzar otro plato.

Stallings (2018), Operating Systems: Internals and Design Principles, 9.ª ed., §2.2 «Procesamiento en serie» · Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.2.1

Lotes y monitor residente

Un encargado que no deja la cocina parada entre cliente y cliente

Entre cliente y cliente, nada

Preparar la cocina a mano para cada cliente tomaba mucho: primero lo hacía el propio cliente; después, un ayudante dedicado. Mientras tanto, la cocina estaba parada.

Recetas en fila

Los clientes dejan sus recetas apiladas y se entregan todas de una vez.

Un encargado que vive en la cocina

Apenas sale un plato, toma la siguiente receta de la pila. Guarda sus notas en un rincón que ningún plato puede tocar y pone un reloj: si un plato tarda demasiado, lo retira.

Pero el horno sigue mandando

Mientras un plato está en el horno, el cocinero lo mira: la cocina sigue quieta.

De la cocina a la CPU

En la cocina	En la computadora
Cliente o ayudante que prepara la cocina	**Preparación manual: primero el programador; después, el operador (años 50). No es parte del SO.**
Recetas apiladas que se entregan juntas	**Procesamiento por lotes**
Encargado que vive en la cocina	**Monitor residente: programa que queda siempre en memoria y carga el trabajo siguiente**
Rincón que ningún plato puede tocar	**Protección de memoria (y modo núcleo)**
Reloj de cocina	**Temporizador: corta al que tarda**

Idea clave

El monitor residente fue el **primer sistema operativo**: un programa que se queda en memoria para encadenar trabajos sin el operador.

Ya trae dos ideas que siguen vigentes: proteger su zona y cortar al que tarda.

Lo que no resuelve: la CPU sigue ociosa mientras espera la E/S.

Stallings (2018), Operating Systems: Internals and Design Principles, 9.ª ed., §2.2 «Sistemas por lotes sencillos» · Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.2.2

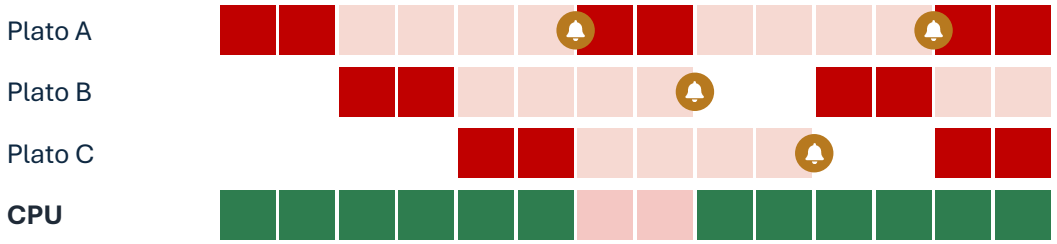
Multiprogramación

Mientras un plato está en el horno, el cocinero avanza otro

Un plato en la cocina



Tres platos en la cocina



El cocinero trabaja en ese plato · El plato está en el horno (E/S) · CPU ocupada · CPU ociosa · El horno avisa: timbre

En la cocina	En la computadora
Varios platos en preparación	**Varios programas en memoria a la vez**
Cambiar de plato cuando uno entra al horno	**El SO aparta al proceso que espera la E/S y da la CPU a otro**
El horno toca el timbre al terminar	**Interrupción de fin de E/S: el SO marca el proceso como listo**
La mesada guarda los tres platos	**Todos deben caber en la RAM (y protegerse)**

Idea clave

Multiprogramar = tener **varios procesos en memoria** para que la CPU siempre tenga algo que hacer.

Funciona gracias a los **timbres** (interrupciones): el horno avisa cuando termina y el administrador no tiene que vigilarlo.

Exige protección de memoria y alguien que reparta turnos: el administrador.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.2.3 «Multiprogramación» y §1.3.5 «Interrupciones» · Stallings (2018), Operating Systems: Internals and Design Principles, 9.ª ed., §2.2 «Sistemas por lotes multiprogramados»

¿Cuánto se aprovecha la CPU?

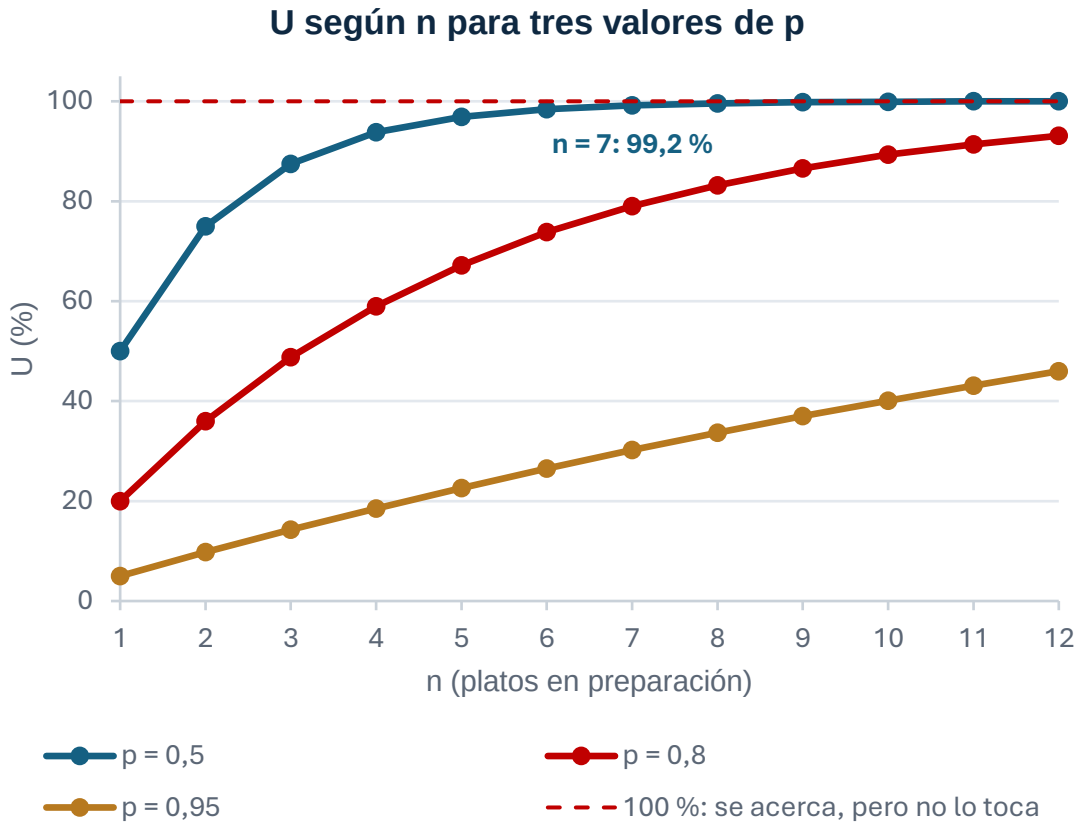
El cocinero solo se queda sin trabajo cuando todos los platos están en el horno a la vez

$$U = 1 - p^n$$

- **p** — **probabilidad de que un plato esté en el horno**: fracción del tiempo que un proceso espera la E/S
- **n** — procesos en memoria (grado de multiprogramación)
- **p^n** — **probabilidad (fracción del tiempo)** de que los n platos estén en el horno a la vez: CPU **ociosa**
- **U** — uso de la CPU (utilización)

Intuición

Si cada plato pasa **la mitad** del tiempo en el horno (p = 0,5): con un plato, el cocinero trabaja el 50 % del tiempo. Con dos, solo se queda sin trabajo en los instantes en que **los dos** están en el horno a la vez: 0,5 × 0,5 = un cuarto de los instantes. Trabaja el **75 %** del tiempo.



Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §2.1.7 «Modelado de la multiprogramación» · Supone esperas independientes: sirve para la tendencia, no para predecir exacto

Ejercicio · ¿Cuánto rinde la cocina?

Cada plato pasa el 80 % del tiempo en el horno

Enunciado

Un programa de oficina espera E/S el 80 % del tiempo (p = 0,8).

1. Calcula U para n = 1, 2, 3 y 4.
2. ¿Cuánto mejora cada proceso añadido?

n	$p^n = 0{,}8^n$	$U = 1 - p^n$	Mejora
1	0,800	**20,0 %**	—
2	0,640	**36,0 %**	+16,0
3	0,512	**48,8 %**	+12,8
4	0,410	**59,0 %**	+10,2

Idea clave

Rendimientos decrecientes: cada plato añadido ayuda menos que el anterior (+16, +12,8, +10,2 puntos).

Con p alto hacen falta muchos procesos… y todos tienen que caber en la mesada (RAM).

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §2.1.7

Tiempo compartido

Muchos clientes en la barra quieren respuesta ya

La barra llena

Veinte clientes esperan, cada uno con su terminal.

Turnos cortos

El jefe de cocina da a cada plato unos milisegundos y pasa al siguiente.

Parece exclusivo

Cada cliente siente que la cocina es solo suya.

Nace el multiusuario

Cuentas, permisos y archivos de cada cliente.

De la cocina a la CPU

En la cocina	En la computadora
Turno corto	**Quantum de tiempo (temporizador)**
Atender a todos un poco	**Tiempo compartido (CTSS, MULTICS, UNIX)**
Clientes con cuenta propia	**Sistema multiusuario**
Jefe de cocina	**Planificador (semana 8)**

Idea clave

Lotes busca aprovechar la CPU; **tiempo compartido** busca responder rápido a cada usuario. Cada tipo de SO nace de un objetivo distinto.

Stallings (2018), Operating Systems: Internals and Design Principles, 9.ª ed., §2.2 «Sistemas de tiempo compartido» · Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.2.3

Tipos de SO según usuarios y tareas

¿Cuántos clientes y cuántos platos a la vez?

	Monotarea	Multitarea
Monousuario	**MS-DOS, el firmware de una lavadora** *Un cliente, un plato a la vez.*	**Android, iOS, Windows en un portátil personal** *Una persona, muchas apps abiertas.*
Multiusuario	**Combinación casi inexistente hoy** *Lotes antiguos: muchos usuarios en fila.*	**Linux, Windows Server, macOS** *Muchas cuentas y muchos procesos a la vez.*

Idea clave

Hoy casi todo es multitarea; la diferencia está en para quién: una persona o muchas cuentas a la vez.

Carretero et al. (2007), Sistemas operativos: una visión aplicada, 2.ª ed., cap. 2 «Tipos de sistemas operativos»

Tipos de SO según su propósito

Cada tipo de restaurante necesita otro administrador

Escritorio

Comedor de casa: un comensal que quiere todo ya. Ej.: Windows 11, macOS.

Servidor

Cocina industrial que atiende comensales todo el día. Ej.: AlmaLinux, Windows Server.

Móvil

Food truck: poco espacio y depende de la batería. Ej.: Android, iOS.

Embebido

Máquina de café: hace una sola cosa, siempre igual. Ej.: router, lavadora.

Tiempo real

Cocina del hospital: la bandeja del paciente debe salir **a la hora exacta**; si sale tarde es un error, aunque esté perfecta. **Ej.: los frenos ABS del auto**: si una rueda se bloquea, deben soltarla en milisegundos, siempre. (SO: QNX, FreeRTOS)

Distribuido

Cadena de restaurantes coordinados como uno. Ej.: clústeres, nube.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.4 «El zoológico de los sistemas operativos» · SO distribuidos y de red: revisión autónoma

Sistemas de tiempo real

La cocina del hospital: la bandeja sale a las 12:00, sí o sí

Plazo fijo

Cada bandeja tiene una hora límite de salida.

Tiempo real duro

Llegar tarde es un fallo grave: marcapasos, frenos ABS, airbag.

Tiempo real blando

Llegar algo tarde molesta pero no es grave: video, VoIP, juegos.

Predecible, no rápido

Importa garantizar el peor caso, no el promedio.

De la cocina a la CPU

En la cocina	En la computadora
Hora límite de la bandeja	**Plazo (deadline)**
Bandeja crítica tarde = fallo	**Tiempo real duro**
Postre algo tarde = molestia	**Tiempo real blando**
Garantizar el peor caso	**Determinismo**

Idea clave

Tiempo real no significa «rápido»: significa **a tiempo y predecible**.

Lo retomamos en planificación (semana 8).

Stallings (2018), Operating Systems: Internals and Design Principles, 9.ª ed., cap. 10 «Planificación de tiempo real» · Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.4.8

Componentes del sistema operativo

Los departamentos del administrador

Procesos

Jefe de cocina: qué plato se cocina y cuándo (los turnos y los timbres de hoy). Semanas 7–11.

Memoria

Reparte la mesada entre los platos (todos deben caber: lo vimos con U). Semanas 12–13.

Entrada/salida

Los meseros: van a la **bodega**, al **salón** y al **proveedor**, y tocan el timbre al volver. Son los que trabajan detrás del horno. Semana 14.

Archivos

El recetario y su archivo (este todavía no lo vimos). Semanas 15–16.

Seguridad

Gafetes y llaves: quién entra a cada zona (el gafete y el rincón del monitor residente). Semana 5.

Red

Comandas con el **proveedor** y con otros restaurantes (el proveedor de la lámina de E/S). Semana 11.

Y una ventanilla para hacer pedidos al administrador: llamadas al sistema y shell (semana 4).

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.5 «Conceptos de los SO» · Carretero et al. (2007), Sistemas operativos: una visión aplicada, 2.ª ed., cap. 2 «Componentes del sistema operativo»

Núcleo y programas de sistema

Quién trabaja con el gafete del administrador y quién no

Aplicaciones

Navegador, Word, juegos

Programas de sistema

shell, ls, systemd, servicios, Administrador de tareas

frontera del gafete: modo usuario | modo núcleo

Núcleo (con gafete)

procesos · memoria · E/S y controladores · archivos · red

Hardware

CPU, RAM, discos, red

En la cocina	En la computadora
Trabajo del administrador, hecho con el gafete puesto	**Núcleo (modo núcleo)**
Recetas de la casa, sin gafete	**Programas de sistema: vienen con el SO (shell, `ls`, systemd)**
Recetas que trae el cliente, sin gafete	**Aplicaciones (navegador, Word)**
Ventanilla de pedidos	**Llamadas al sistema (sem. 4)**

Idea clave

El SO es más que el núcleo, pero **solo el código del administrador (el núcleo) se ejecuta con el gafete**.

Los demás programas piden en la ventanilla.

1. `ls` pide en la ventanilla
2. el cocinero se pone el gafete y hace el trabajo del administrador
3. se lo quita y le entrega la lista a `ls`

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.3 y §1.7 · Carretero et al. (2007), Sistemas operativos: una visión aplicada, 2.ª ed., cap. 2

Quiz de la sesión 1 · Pregunta 1 de 2

Responde antes de pasar la lámina

Con p = 0,5 y n = 3 procesos, ¿cuál es el uso de la CPU?

A 50 %

B 75 %

C 87,5 %

D 150 %

Quiz de la sesión 1 · Respuesta 1 de 2

Pregunta: Con p = 0,5 y n = 3 procesos, ¿cuál es el uso de la CPU?

A) 50 %

B) 75 %

C) 87,5 %

D) 150 %

Una cuenta de una línea

$U = 1 - 0{,}5^3 = 1 - 0{,}125 =$ **0,875**.

La CPU solo está ociosa cuando **los tres están en el horno** a la vez: $0{,}5^3 = 12{,}5$ % del tiempo.

D es imposible: U nunca supera el 100 % (ni lo alcanza). B suma 25 % por proceso: el modelo no es lineal.

Quiz de la sesión 1 · Pregunta 2 de 2

Responde antes de pasar la lámina

Un marcapasos debe generar el pulso en menos de 1 ms, siempre. ¿Qué tipo de SO necesita?

A Tiempo real duro

B Tiempo real blando

C Tiempo compartido

D Por lotes

Quiz de la sesión 1 · Respuesta 2 de 2

Pregunta: Un marcapasos debe generar el pulso en menos de 1 ms, siempre. ¿Qué tipo de SO necesita?

A) Tiempo real duro

B) Tiempo real blando

C) Tiempo compartido

D) Por lotes

Llegar tarde es un fallo

Si un pulso llega tarde, el paciente corre riesgo: es **tiempo real duro**.
Blando sería un video que se congela un instante.
Tiempo compartido y lotes buscan otros objetivos (respuesta promedio y aprovechamiento), no garantizar plazos.

Cierre de la sesión 1 (Puntos clave)

El administrador y por qué hace falta

1. **El SO es el administrador:** no cocina; simplifica (máquina extendida), reparte (gestor de recursos) **y protege**.

No cocina: organiza.

2. **La multiprogramación nació para que la CPU no se quede sin trabajo mientras los procesos esperan la E/S:** $U = 1 - p^n$, con rendimientos decrecientes y **sin llegar nunca al 100 %**.

Mientras un plato está en el horno, el cocinero avanza otro.

3. **Cada tipo de SO responde a un objetivo:** lotes, tiempo compartido, tiempo real, móvil, embebido.

Cada restaurante necesita su administrador.

4. **Seis departamentos; solo el código del administrador (el núcleo) se ejecuta con el gafete:** los programas piden en la ventanilla.

Los departamentos del administrador.

Ahora · Taller 1 «Conoce a tu administrador»: en parejas, uno en la VM AlmaLinux y otro en Windows. Abran la guía en la sección del Taller 1.

Taller 1 · Conoce a tu administrador

Verlo trabajar y medir cuánto espera un plato

Objetivo

Encontrar a cada departamento del administrador en un equipo real y medir cuánto espera un programa.

Producto

Hoja de resultados de la pareja: tabla de departamentos (Linux | Windows), p medido y número de procesos para U ≥ 90 %.

Tiempo · 55 min

6 min de explicación · ≈ 45 min en parejas · 4 min de puesta en común.

Material

Guía de talleres (sección Taller 1) · VM AlmaLinux · Windows con PowerShell · Laboratorio del Administrador (recurso interactivo).

Taller 1 · Pasos y conceptos

Lo que aprenderán mientras trabajan

A · Departamentos en vivo

15 min

- Linux: `ps`, `pstree`, `free -h`, `lsblk`, `/proc`, `/dev`, `vmstat`
- Windows: `Get-Process`, `Get-Service`, `Get-CimInstance`

CONCEPTOS

- **PID y proceso 1**
- **Servicio (daemon)**
- **/proc, ventana al núcleo**
- **/dev: dispositivos como archivos**
- **CPU ociosa: id frente a wa**

B · Mide la espera

12 min

- Linux: `time read` (espera el teclado), un bucle de cálculo y una búsqueda en disco
- Windows: `TotalProcessorTime` frente a tiempo real

CONCEPTOS

- **Tiempo real, de usuario y de sistema**
- **Limitado por CPU frente a limitado por E/S**

C · Multiprogramación

18 min

- Juntos, con el artefacto: su p medido, la curva U(n) y la línea de tiempo

CONCEPTOS

- **La mesada limita la multiprogramación**

Taller 1 · Parejas, reto y criterios de éxito

Linux | Windows en paralelo; mañana se invierten los roles

Integrante 1 · Linux

Trabaja en la VM AlmaLinux. En el Taller 2 pasa a Windows.

Integrante 2 · Windows

Trabaja en PowerShell. En el Taller 2 pasa a Linux.

Tabla comparativa

En cada parte llenan juntos la tabla Linux | Windows y se explican su columna.

Reto (parte C)

¿Cuántos procesos para U ≥ 90 % con su p? ¿Caben en 8 GiB si el SO usa 2 GiB y cada uno 512 MiB?

Criterios de éxito

Tabla completa en ambos sistemas · p con 3 decimales · reto justificado.

Si no terminan

La guía trae puntos de control con tiempo sugerido. Lo pendiente se completa en casa.

Sesión 2 · ¿Cómo se organiza y cómo abre el local?

Teoría 55 min (temas 2.2 y 2.3) + Taller 2 «Abre y cierra el local» 55 min

Lo que quedó pendiente

Ya conocemos al administrador; falta ver su oficina y su rutina

¿Todos con gafete?

¿Todos los departamentos trabajan dentro de la zona restringida? → Estructura del núcleo.

¿Quién lo despierta?

¿Cómo toma el control cada mañana? → Arranque en 5 pasos.

¿Cómo cierra?

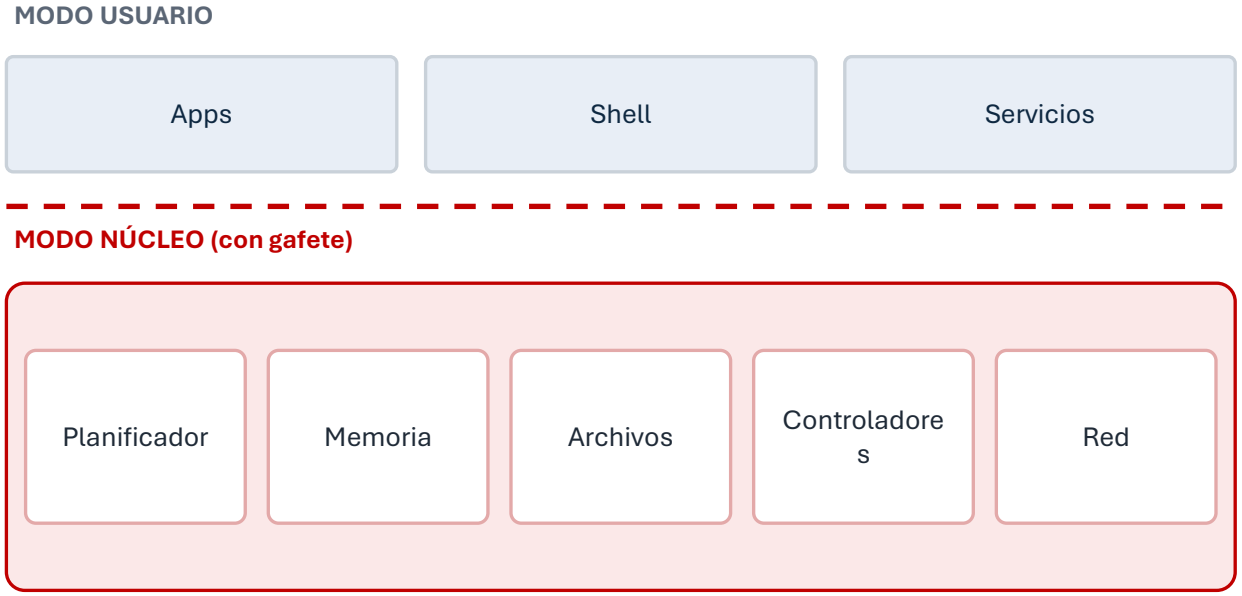
¿Cómo se apaga sin perder nada? → Parada ordenada.

Recordemos

Pregunta de la semana: ¿quién es el administrador, qué hace y cómo toma el control de la cocina cada mañana? Hoy respondemos la segunda mitad.

Núcleo monolítico

Todos los departamentos en una sola oficina blindada



Un solo programa grande en modo núcleo. Ej.: Linux, FreeBSD.

En la cocina	En la computadora
Una oficina con todos adentro	**Núcleo monolítico**
Se hablan de escritorio a escritorio	**Llamadas a funciones (rápido)**
Un empleado se desmaya y para todo	**Kernel panic / pantalla azul**

Idea clave

Ventaja: es rápido; todo está a una llamada de distancia.

Riesgo: un error en cualquier parte, por ejemplo un controlador, tumba todo el sistema.

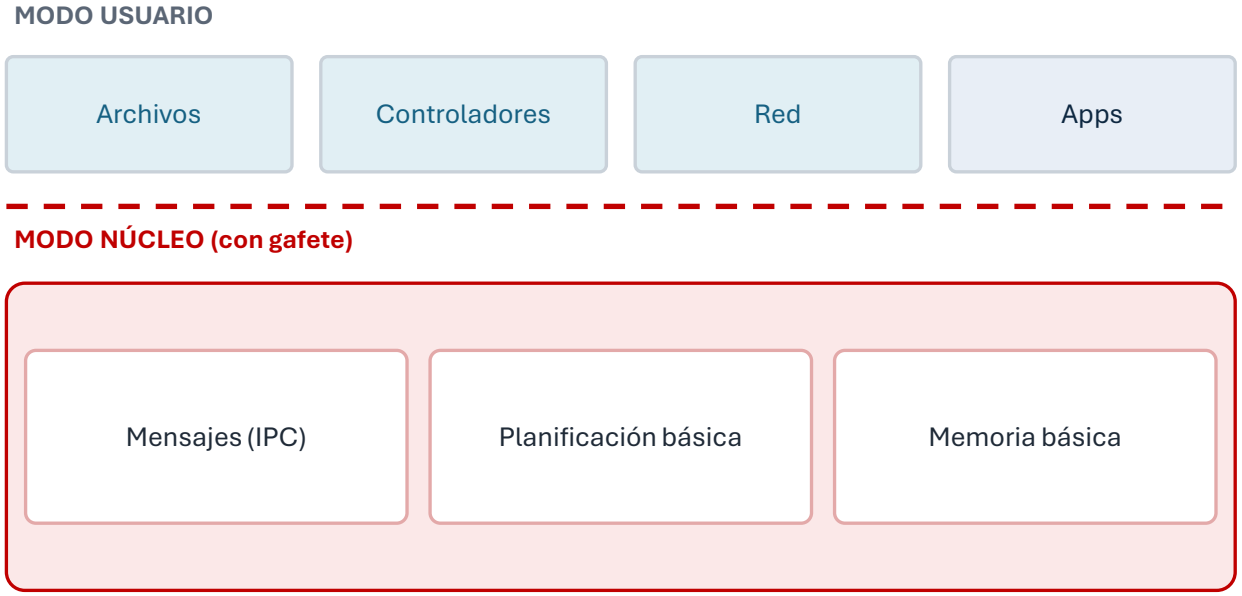
En AlmaLinux

Linux es monolítico: el núcleo es un único programa que corre completo con el gafete.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.7.1 «Sistemas monolíticos»

Microkernel

Oficina mínima; los departamentos trabajan afuera y se pasan notas



En la cocina	En la computadora
Oficina con lo mínimo	**Microkernel**
Departamentos fuera de la oficina	**Servidores en modo usuario**
Se pasan notas por la ventanilla	**Mensajes (IPC)**

Idea clave

Ventaja: si un controlador falla, se reinicia y el resto sigue.

Costo: cada nota cruza la frontera del gafete: más lento.

Ej.: QNX (autos, equipos médicos), MINIX 3, seL4.

Dónde se usa

Se elige donde un fallo no puede tumbar el sistema: frenos, aviones, equipos médicos.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.7.3 «Microkernels» · Tanenbaum es autor de MINIX

Híbridos y módulos

Oficinas mixtas y especialistas que se contratan sobre la marcha

Núcleos híbridos

- **Windows (NT):** núcleo, ejecutivo y controladores en modo núcleo; subsistemas en modo usuario.
- **macOS (XNU):** Mach + BSD juntos en modo núcleo.
- En la práctica se comportan casi como monolíticos: un controlador (.sys) defectuoso aún causa una pantalla azul.
- Detalle: revisión autónoma.

Módulos: especialistas contratados

En la cocina	En la computadora
El especialista	**Módulo (.ko) · controlador**
Contratarlo	`modprobe nombre`
Despedirlo	`rmmod nombre`
Lista del personal	`lsmod · driverquery`

Linux es **monolítico pero modular**: se cargan en caliente, pero corren **con gafete** (modo núcleo).

Lo practicarán en el Taller 2 con `modprobe dummy`.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §1.7 y cap. 11 · Red Hat (2025), RHEL 10: Managing, monitoring and updating the kernel

Abrir el local: el arranque en 5 pasos

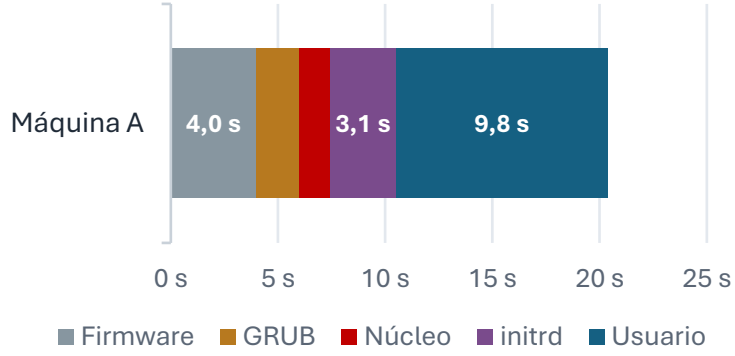
Del botón de encendido a la ventanilla abierta

1 · Firmware	2 · Gestor de arranque	3 · Núcleo + initramfs	4 · PID 1	5 · Login y shell
UEFI/BIOS + POST	**GRUB · Windows Boot Manager**	**vmlinuz · ntoskrnl.exe**	**systemd · smss → services**	**gdm/getty · winlogon**
El guardia revisa luz y gas	*El cartel: a qué administrador llamar*	*Llega el administrador con su maletín*	*Se encienden las estaciones*	*Se abre la ventanilla*

Idea clave

Cada paso **carga al siguiente y le pasa el control**.

Si uno falla, el arranque se detiene justo ahí: por eso el mensaje en pantalla dice en qué etapa estamos.



systemd-analyze(1) y bootup(7), freedesktop.org · Carretero et al. (2007), Sistemas operativos: una visión aplicada, 2.ª ed., cap. 2 «Arranque del sistema operativo» · Máquina A: VM AlmaLinux 10.2 (firmware y GRUB medidos con cronómetro)

Paso 1 · Firmware: UEFI/BIOS y POST

El guardia del edificio revisa luz, gas y agua

Se pulsa el botón

La CPU arranca en una dirección fija del chip de firmware.

POST

Prueba de encendido: RAM, teclado, discos.

Busca un disco

Sigue el orden de arranque guardado en la placa.

Entrega el control

Carga el gestor de arranque y le pasa la CPU.

De la cocina a la CPU

En la cocina	En la computadora
El guardia	**Firmware (UEFI o BIOS)**
Revisar luz y gas	**POST (Power-On Self-Test)**
Lista de a quién llamar	**Orden de arranque (NVRAM)**
Nadie a quien llamar	**«No bootable device»**

Idea clave

El firmware **no es el SO**: vive en la placa base y solo prepara el terreno.

UEFI lee una partición especial (ESP); la BIOS antigua, el primer sector del disco (MBR).

Carretero et al. (2007), Sistemas operativos: una visión aplicada, 2.ª ed., cap. 2 · UEFI Forum, «UEFI Specification» · MBR frente a GPT: revisión autónoma

Paso 2 · Gestor de arranque

El cartel que dice a qué administrador llamar

Muestra el menú

GRUB lista los núcleos instalados (y Windows, si hay arranque dual).

Elige uno

Por defecto, el más reciente. En AlmaLinux el menú está oculto: Shift o Esc.

Carga dos archivos

vmlinuz (el núcleo) e initramfs (su maletín) en la RAM.

Pasa instrucciones

Parámetros como root=… y quiet; luego salta al núcleo.

De la cocina a la CPU

En la cocina	En la computadora
El cartel	**GRUB 2 · Windows Boot Manager**
Varios administradores posibles	**Varios núcleos o arranque dual**
Instrucciones del cartel	**Línea de comandos del núcleo**
Cartel borrado	**«grub rescue>»**

Idea clave

El gestor de arranque es un programa pequeño cuya única tarea es **cargar el núcleo y pasarle el control.**

Las instrucciones que recibió el núcleo se ven en `/proc/cmdline`.

GNU GRUB Manual 2.12 · Red Hat (2025), RHEL 10: Managing, monitoring and updating the kernel · Windows: bootmgr, winload.efi y BCD (bcdedit)

Paso 3 · Núcleo e initramfs

Llega el administrador con un maletín mínimo

Se instala

Detecta CPU, memoria y dispositivos; prepara sus estructuras.

Abre el maletín

initramfs: un sistema de archivos mínimo en RAM con los controladores justos.

Llega a la bodega

Encuentra el disco raíz (LVM, cifrado…) y lo monta.

Lanza el proceso 1

Ejecuta systemd. El núcleo no lanza ningún otro proceso así.

De la cocina a la CPU

En la cocina	En la computadora
El maletín mínimo	**initramfs (dracut en AlmaLinux)**
Llegar a la bodega	**Montar el sistema de archivos raíz /**
El primer encargo	**PID 1 (systemd)**
Sin maletín	**«Kernel panic… unable to mount root fs»**

Idea clave

¿Por qué un maletín? El núcleo no trae todos los controladores; sin initramfs necesitaría el disco para leer los controladores del disco.

En Windows: ntoskrnl.exe y los controladores de arranque.

Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §10.2 · dracut(8) · Carretero et al. (2007), Sistemas operativos: una visión aplicada, 2.ª ed., cap. 2

Pasos 4 y 5 · PID 1 y login

Se encienden las estaciones y se abre la ventanilla

PID 1

systemd, el primer proceso y padre de todos (pstree).

Unidades en paralelo

Servicios y montajes con dependencias; lo independiente arranca a la vez.

Target

multi-user.target (servidor) o graphical.target (escritorio).

Login y shell

Aparece el login; tras entrar, la shell o el escritorio: la ventanilla.

De la cocina a la CPU

En la cocina	En la computadora
Las estaciones	**Servicios (daemons)**
Qué necesita cada una	**Unidades y dependencias**
Local listo	**Target alcanzado → login**
En Windows	**smss → services.exe → winlogon**

Idea clave

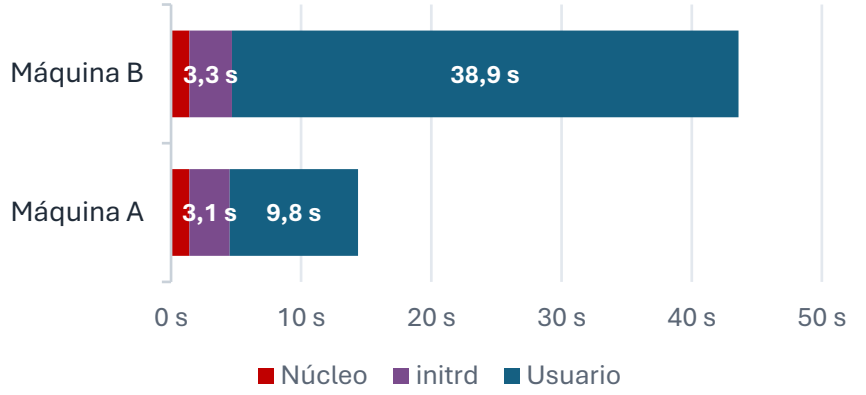
`systemd-analyze` mide este paso: cuánto tardó cada unidad (`blame`) y cuál hizo esperar a todas (`critical-chain`).

systemd-analyze(1) y bootup(7), freedesktop.org · Windows: «Boot process», Microsoft Learn

Ejercicio del profesor · ¿Por qué tarda B?

Encontrar la estación que hace esperar a todo el local

```
$ systemd-analyze
Startup finished in 1.398s (kernel) + 3.287s (initrd)
                 + 38.912s (userspace) = 43.597s
$ systemd-analyze blame --no-pager | head -3
25.021s mnt-respaldo.mount
 6.118s NetworkManager-wait-online.service
 2.296s dnf-makecache.service
$ systemd-analyze critical-chain --no-pager
graphical.target @38.876s
└─multi-user.target @38.875s
  └─remote-fs.target @38.870s
    └─mnt-respaldo.mount @13.846s +25.021s
      └─network-online.target @13.838s
        └─NetworkManager-wait-online.service +6.118s
```



Preguntas

1. ¿Qué etapa explica la diferencia?
2. ¿Qué unidad es la culpable?
3. ¿Por qué NetworkManager-wait-online sí importa aquí?
4. ¿Cómo lo arreglarías?

Máquina B: muestras_logs_arranque.txt, sección 2 · systemd-analyze(1) y bootup(7), freedesktop.org

Ejercicio · Solución

blame dice quién tardó; critical-chain, quién hizo esperar a todos

1. La diferencia está en **espacio de usuario**: 38,9 s frente a 9,8 s; núcleo e initrd son casi iguales.

2. Culpable: **mnt-respaldo.mount**, 25 s montando una carpeta de red (NFS) de un servidor que no responde.

3. Está en la **cadena crítica**: remote-fs.target lo espera, y el montaje espera a la red (6,1 s de wait-online).

4. Arreglo: reparar el servidor o montar bajo demanda en /etc/fstab: `nofail,_netdev,x-systemd.automount`. Arranque esperado: ≈ 12–14 s.

```
192.168.56.20:/srv/respaldo  /mnt/respaldo  nfs  defaults,nofail,_netdev,x-systemd.automount  0 0
```

systemd.mount(5), opciones x-systemd.automount y nofail

Cerrar el local: la parada ordenada

No se apaga la luz con gente cocinando

Avisar

SIGTERM: «cerramos, terminen sus platos».

Esperar y forzar

Unos segundos de cortesía; luego SIGKILL a los rezagados.

Guardar en la bodega

sync: lo que quedó en la mesada se escribe en el disco; se desmonta.

Apagar la luz

ACPI apaga la máquina o la reinicia.

De la cocina a la CPU

En la cocina	En la computadora
Aviso amable	**SIGTERM (señal 15)**
Sacar a los rezagados	**SIGKILL (señal 9)**
Llevar todo a la bodega	**sync + desmontar**
Apagar o reiniciar	**`systemctl poweroff · reboot`**

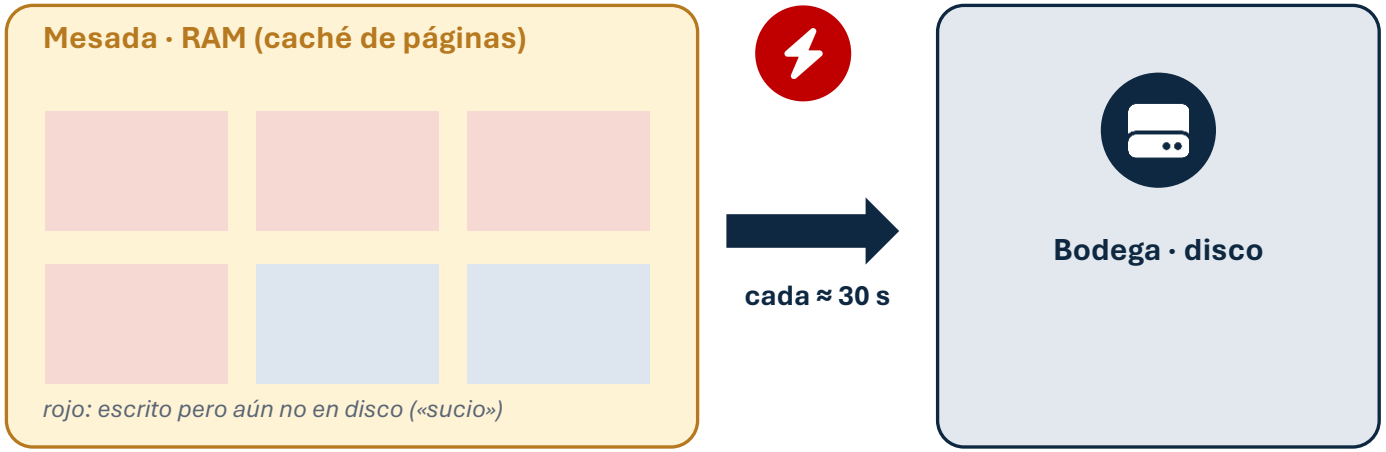
Idea clave

La parada es **el arranque al revés**: systemd detiene las unidades en orden inverso a sus dependencias. Las señales se estudian en la semana 7.

systemd-analyze(1) y bootup(7), freedesktop.org · systemd-halt.service(8) · Tanenbaum y Bos (2023), Modern Operating Systems, 5.ª ed., §10.3 (señales)

¿Por qué un corte de energía corrompe datos?

Lo que estaba en la mesada nunca llegó a la bodega



¿Cuánto se arriesga?

Si un programa escribe 20 MB/s y se vacía cada 30 s, hasta **600 MB** solo están en la RAM.

¿Qué se daña?

Archivos recientes y, a veces, la estructura del sistema de archivos a medio escribir.

¿Cómo se mitiga?

Apagado ordenado, `sync`, UPS y sistemas de archivos con diario (semana 15).

Idea clave

Escribir en la RAM y llevarlo al disco después es rápido (semana 2: la bodega está lejos).
El precio es una **ventana de riesgo**: si se va la luz, lo que estaba en la mesada se pierde.

Documentación del núcleo Linux, sysctl vm: dirty_expire_centisecs = 3000 (30 s) por defecto · Laboratorio del Administrador, pestaña 2

Quiz de la sesión 2 · Pregunta 1 de 2

Responde antes de pasar la lámina

¿Qué proceso tiene el PID 1 en AlmaLinux?

A GRUB

B systemd

C bash

D El núcleo

Quiz de la sesión 2 · Respuesta 1 de 2

Pregunta: ¿Qué proceso tiene el PID 1 en AlmaLinux?

A) GRUB

B) systemd

C) bash

D) El núcleo

El primer encargo del administrador

El núcleo lanza un único proceso: **systemd**, con PID 1. Es el padre de todos (pstree).

GRUB no es un proceso: termina su trabajo antes de que exista el núcleo.

El núcleo no es un proceso con PID; bash aparece mucho después, tras el login.

Quiz de la sesión 2 · Pregunta 2 de 2

Responde antes de pasar la lámina

¿Para qué sirve el initramfs?

A Es el gestor de arranque

B Un sistema de archivos mínimo en RAM para encontrar y montar la raíz

C Guarda la configuración del firmware

D Es el primer proceso del sistema

Quiz de la sesión 2 · Respuesta 2 de 2

Pregunta: ¿Para qué sirve el initramfs?

A) Es el gestor de arranque

B) Un sistema de archivos mínimo en RAM para encontrar y montar la raíz

C) Guarda la configuración del firmware

D) Es el primer proceso del sistema

El maletín del administrador

El **initramfs** lleva los controladores y herramientas justos para encontrar el disco raíz (LVM, cifrado…) y montarlo.
Sin él: «Kernel panic… unable to mount root fs».
En AlmaLinux lo genera dracut.

Cierre de la sesión 2 (Puntos clave)

La oficina del administrador y su rutina

1. **Estructura del núcleo:** monolítico (rápido, frágil), microkernel (robusto, más lento), híbrido; Linux es monolítico y modular.
2. **Arranque en 5 pasos:** firmware → gestor de arranque → núcleo + initramfs → PID 1 → login.
3. **systemd-analyze:** `blame` dice quién tardó; `critical-chain`, quién hizo esperar a todos.
4. **Parar bien importa:** SIGTERM → SIGKILL → sync → apagar; un corte deja datos en la mesada.

La oficina del administrador.

Abrir el local.

La estación lenta.

Cerrar el local.

Ahora · Taller 2 «Abre y cierra el local»: roles invertidos. Quien tenga laptop con Windows, ábrela con PowerShell como administrador.

Taller 2 · Abre y cierra el local

Medir, diagnosticar y controlar el arranque y la parada

Objetivo

Medir el arranque de su propia máquina, encontrar la etapa más lenta y programar un apagado sin perder nada.

Producto

Tabla de tiempos de arranque (Linux | Windows) y el esqueleto del Deber 3 con las secciones 1–3, subido al GitHub de la pareja.

Tiempo · 55 min

6 min de explicación · ≈ 45 min en parejas · 4 min de puesta en común.

Material

Guía (Taller 2) · VM AlmaLinux · laptop con Windows (PowerShell como administrador) · muestras_logs_arranque.txt · informe_arranque_base.sh/.ps1.

Taller 2 · Pasos y conceptos

Roles invertidos: quien estuvo en Linux pasa a Windows

A · Observa tu arranque

14 min

- Linux: `systemd-analyze`, `blame`, `critical-chain`, `journalctl -b`
- Windows: `LastBootUpTime`, evento 100

CONCEPTOS

- **Unidad y target**
- **Registro del sistema**

B · Módulos

10 min

- Juntos en Linux: `lsmod`, `modinfo`, `modprobe dummy`, `rmmod`
- Comparar con `driverquery`

CONCEPTOS

- **Módulo / controlador**

C · Cierra el local

8 min

- Programar y cancelar un reinicio
- Windows: `powercfg /a` y el inicio rápido

CONCEPTOS

- **Inicio rápido**
- **Apagado programado**

D · Deber 3

13 min

- Secciones 1–3 de `informe_arranque_base`
- Primer commit en GitHub

CONCEPTOS

- **Script de administración**

Taller 2 · Laptops, reto y entrega

Lo que necesita permisos se hace en la laptop

Laptop con Windows

PowerShell como administrador solo para el evento 100 (desglose del arranque).

Sin laptop

Usen muestras_logs_arranque.txt, sección 3: mismos datos.

Reto (parte A)

Comparen su arranque con la máquina B de las muestras: ¿qué unidad optimizarían y por qué?

Entrega

semana03/informe_arranque.sh y .ps1 · commit de cada integrante · lo pendiente, en casa.

Criterios de éxito

Tabla de tiempos llena · reto respondido · script corre sin errores · commit hecho.

Cuidado

No reinicien el equipo del laboratorio: programen y cancelen.

Taller 2 · Puesta en común

Y el gancho hacia la semana 4



¿Cuál fue la etapa más lenta de su arranque, en Linux y en Windows?

Casi siempre userspace o posarranque.



¿Qué hizo `modprobe dummy` y con qué gafete corre ese módulo?

Añadió un controlador al núcleo en marcha; corre en modo núcleo.



¿Por qué «Apagar» no cambió LastBootUpTime con inicio rápido?

El núcleo se hibernó y se recuperó.

El local está abierto y la ventanilla también: ¿cómo se le hacen pedidos al administrador? → Semana 4

Sesión 3 · Consolidación

Consolidación · resumen, 4 ejercicios, quiz final y Deber 3 · 50 min

La semana en una lámina

¿Quién es el administrador, qué hace y cómo toma el control cada mañana?



Por qué existe

Para que la CPU no se quede sin trabajo durante la E/S: multiprogramación, $U = 1 - p^n$.



Qué tipos hay

Lotes, tiempo compartido, tiempo real, móvil, embebido, servidor.



Cómo se organiza

Seis departamentos; núcleo monolítico, microkernel o híbrido; módulos.



Cómo abre y cierra

Arranque en 5 pasos; parada ordenada; inicio rápido.

Hoy: repasamos en 10 minutos, resolvemos 4 ejercicios y cerramos con un quiz de 5 preguntas.

Resumen 1 · Por qué existe y qué tipos hay

Cada tipo de restaurante responde a un objetivo

Serie
La CPU espera la E/S

Lotes
Monitor residente

Multiprogramación
$U = 1 - p^n$

Tiempo compartido
Respuesta rápida

Tipo	Qué le importa
Escritorio · móvil	Respuesta al usuario · batería
Servidor	Muchos clientes, estabilidad
Embebido	Una función, pocos recursos
Tiempo real duro / blando	Plazos garantizados / casi siempre

Idea clave

$U = 1 - p^n$: la CPU solo se queda sin trabajo cuando los n platos están en el horno a la vez; nunca llega al 100 %.

Rendimientos decrecientes y la RAM como límite.

Medido en el taller: p = 1 − (user + sys) / real.

Resumen 2 · Cómo se organiza

Departamentos, oficina y especialistas

Departamentos

Procesos, memoria, E/S, archivos, seguridad, red. Solo el código del núcleo se ejecuta con el gafete.

Monolítico

Todos en la oficina blindada: rápido; un error tumba todo (Linux).

Microkernel

Oficina mínima y mensajes: robusto, más lento (QNX, MINIX 3).

Módulos

Especialistas en caliente: `lsmod`, `modprobe`, `rmmod`. Corren con gafete.

Resumen 3 · Abrir el local

Cinco pasos y tres comandos para medirlos



Comando	Qué responde
`systemd-analyze`	Cuánto tardó cada etapa (kernel, initrd, userspace)
`systemd-analyze blame · critical-chain`	Quién tardó · quién hizo esperar a todos
`journalctl -b · dmesg`	Qué pasó desde que arrancó el núcleo (no firmware ni GRUB)
`LastBootUpTime` · **evento 100**	Último arranque y desglose en Windows

Resumen 4 · Cerrar el local

Avisar, esperar, guardar y apagar

SIGTERM

Avisar

SIGKILL

Forzar a los rezagados

sync

Mesada → bodega

Apagar

ACPI

Corte de energía

Se pierde lo que estaba en la mesada: hasta ritmo × intervalo de vaciado.

Inicio rápido

«Apagar» hiberna el núcleo: LastBootUpTime no cambia. Reiniciar sí.

Programar y cancelar

`shutdown -r +5` / `-c` · `shutdown /r /t 300` / `/a`.

Ejercicio 1 · El servidor que no da abasto

U con la mesada limitada

Datos: servidor con 16 GiB de RAM; el SO usa 4 GiB; cada proceso de la aplicación ocupa 1,5 GiB y espera la base de datos el 90 % del tiempo (p = 0,9).

a) ¿Cuántos procesos caben en la RAM?

b) ¿Qué uso de la CPU se logra con ellos?

c) ¿Cuántos harían falta para U ≥ 90 %?

d) ¿Qué conviene comprar: una CPU más rápida o más RAM? ¿Por qué?

Fórmulas

$n_{max} = \lfloor (RAM - SO) / tamaño \rfloor$

$U = 1 - p^n$

$n \geq \ln(1 - U) / \ln p$

Pueden comprobarlo en el artefacto, pestaña 1, escenario C con estos datos.

Ejercicio 1 · Solución

El cuello de botella es la mesada

a (16 − 4) GiB / 1,5 GiB = **8 procesos**.

b $U = 1 - 0{,}9^{8} = 1 - 0{,}430 =$ **57 %**.

c $n \geq \ln(0{,}1) / \ln(0{,}9) = 21{,}9 \rightarrow$ **22 procesos** (33 GiB solo para ellos).

d **Más RAM.** La CPU ya está ociosa el 43 % del tiempo: hacerla más rápida no ayuda. Con más memoria caben más procesos (o reducir p con un disco más rápido).

Comprobación: $0{,}9^{8} = 0{,}4305 \cdot \ln(0{,}1) = -2{,}3026 \cdot \ln(0{,}9) = -0{,}1054$

Ejercicio 2 · El servidor que no arranca

¿En qué etapa se quedó cada equipo?

1

```
Booting from Hard Disk...

No bootable device -- insert boot
disk and press any key
```

2

```
error: unknown filesystem.
Entering rescue mode...
grub rescue> _
```

3

```
dracut-initqueue[389]: Warning:
/dev/mapper/almalinux-root does not exist
Entering emergency mode.
dracut:/# _
```

4

```
[FAILED] Failed to mount mnt-respaldo.mount
[DEPEND] Dependency failed for remote-fs.target
You are in emergency mode.
Give root password for maintenance
```

Etapas posibles: firmware · gestor de arranque · núcleo · initrd · espacio de usuario. Para cada pantalla: etapa y qué harías.

Ejercicio 2 · Solución

El que habla en la pantalla dice dónde se quedó

Pantalla	Etapa	Pista	Qué harías
1	**Firmware**	Habla la BIOS: no encontró disco	Revisar orden de arranque y disco
2	**Gestor de arranque**	Prompt de GRUB	Reinstalar o reconfigurar GRUB
3	**initrd**	Prompt dracut:/#	Revisar root= y el volumen LVM
4	**Espacio de usuario**	Mensajes de systemd	journalctl -xb y corregir /etc/fstab (nofail)

Idea clave

Firmware y GRUB no dejan rastro en `journalctl`: hay que mirar la pantalla. Desde el núcleo en adelante, el registro del sistema lo cuenta.

Ejercicio 3 · El apagón

Lo que estaba en la mesada

Situación: una base de datos escribe 40 MB/s. El sistema vacía la caché de escritura cada 30 s. Se va la luz sin aviso.

a) ¿Cuántos datos pueden perderse, como máximo?

b) ¿Qué paso de la parada ordenada lo habría evitado?

c) ¿Por qué no vaciar la caché cada segundo?

d) Además del apagado ordenado, ¿qué protege contra esto?



Pueden simularlo en el artefacto, pestaña 2, «Apagado ordenado o corte de energía».

Ejercicio 3 · Solución

Rapidez a cambio de una ventana de riesgo

a 40 MB/s × 30 s = **hasta 1 200 MB**.

b **sync** (guardar en la bodega) dentro del apagado ordenado; antes, SIGTERM deja que la base de datos cierre bien.

c Escribir en el disco es lento: agrupar escrituras da rendimiento. Es un equilibrio entre **rapidez y riesgo**.

d **UPS** (batería) para apagar en orden, y sistemas de archivos con diario y bases de datos con registro de transacciones (semana 15).

Ejercicio 4 · El controlador defectuoso

El especialista se desmaya dentro de la oficina

Situación: el controlador de una tarjeta Wi-Fi tiene un error y accede a una dirección de memoria inválida.

a) ¿Qué pasa en Linux (monolítico)?

b) ¿Y en Windows?

c) ¿Y en QNX o MINIX 3 (microkernel)?

d) Entonces, ¿por qué Linux y Windows no son microkernel?

Monolítico

Controlador dentro de la oficina blindada (modo núcleo).

Microkernel

Controlador fuera, como un proceso más (modo usuario).

Ejercicio 4 · Solución

Dónde trabaja el especialista decide cuánto daño hace

a Linux: el error ocurre en modo núcleo → «oops» o **kernel panic**; puede caer todo el sistema.

b Windows: también en modo núcleo (.sys) → **pantalla azul** que suele nombrar al controlador.

c Microkernel: el controlador es un proceso de usuario → se **reinicia** y el resto sigue funcionando.

d **Rendimiento:** cada mensaje cruza la frontera del gafete. Linux compensa con módulos y Windows con su diseño híbrido.

Quiz final · Pregunta 1 de 5

Responde antes de pasar la lámina

¿Qué tipo de SO prioriza responder rápido a muchos usuarios interactivos?

A Por lotes

B Tiempo compartido

C Tiempo real duro

D Embebido

Quiz final · Respuesta 1 de 5

Pregunta: ¿Qué tipo de SO prioriza responder rápido a muchos usuarios interactivos?

A) Por lotes

B) Tiempo compartido

C) Tiempo real duro

D) Embebido

Turnos cortos para todos

Tiempo compartido: turnos de milisegundos para que cada usuario sienta la máquina como propia.
Lotes busca aprovechar la CPU; tiempo real, cumplir plazos.

Quiz final · Pregunta 2 de 5

Responde antes de pasar la lámina

Con n = 4 fijo, si p sube de 0,5 a 0,9, ¿qué pasa con U?

A Sube

B Baja de 93,8 % a 34,4 %

C No cambia

D Llega a 0

Quiz final · Respuesta 2 de 5

Pregunta: Con n = 4 fijo, si p sube de 0,5 a 0,9, ¿qué pasa con U?

A) Sube

B) Baja de 93,8 % a 34,4 %

C) No cambia

D) Llega a 0

Más espera, más CPU ociosa

$1 - 0{,}5^4 = 0{,}938$ y $1 - 0{,}9^4 = 0{,}344$: **baja**.
Con procesos que esperan mucho hacen falta muchos más para aprovechar la CPU.

Quiz final · Pregunta 3 de 5

Responde antes de pasar la lámina

En Linux, `sudo modprobe dummy`…

A recompila el núcleo

B añade un módulo al núcleo en marcha, que corre en modo núcleo

C lo convierte en microkernel

D inicia un proceso de usuario

Quiz final · Respuesta 3 de 5

Pregunta: En Linux, `sudo modprobe dummy`…

A) recompila el núcleo

B) añade un módulo al núcleo en marcha, que corre en modo núcleo

C) lo convierte en microkernel

D) inicia un proceso de usuario

Contratar a un especialista

Carga el módulo en caliente; corre **con gafete** (modo núcleo). Por eso un módulo defectuoso puede tumbar el sistema.

Quiz final · Pregunta 4 de 5

Responde antes de pasar la lámina

¿Cuál es el orden correcto del arranque?

A GRUB → firmware → núcleo → systemd

B Firmware → GRUB → núcleo + initramfs → systemd → login

C Núcleo → firmware → GRUB → login

D Firmware → systemd → GRUB → núcleo

Quiz final · Respuesta 4 de 5

Pregunta: ¿Cuál es el orden correcto del arranque?

A) GRUB → firmware → núcleo → systemd

B) Firmware → GRUB → núcleo + initramfs → systemd → login

C) Núcleo → firmware → GRUB → login

D) Firmware → systemd → GRUB → núcleo

Cada paso carga al siguiente

El firmware carga el gestor de arranque; este, el núcleo y su initramfs; el núcleo lanza systemd (PID 1); al final, el login.

Quiz final · Pregunta 5 de 5

Responde antes de pasar la lámina

Con inicio rápido activo, tras «Apagar» y encender, LastBootUpTime…

A cambia a la hora actual

B no cambia: el núcleo se recuperó hibernado

C se borra

D muestra la hora del firmware

Quiz final · Respuesta 5 de 5

Pregunta: Con inicio rápido activo, tras «Apagar» y encender, LastBootUpTime…

A) cambia a la hora actual

B) no cambia: el núcleo se recuperó hibernado

C) se borra

D) muestra la hora del firmware

Apagar no es reiniciar

«Apagar» hiberna el núcleo; al encender lo recupera, así que **no cambia**.
Para un arranque completo: Reiniciar o `shutdown /s /full`.

Deber 3 · Informe de arranque

Script en Bash y PowerShell · en parejas · portafolio de GitHub

Sección	Linux	Windows	Dónde
1 · SO, versión y núcleo	`/etc/os-release`, `uname`	`Win32_OperatingSystem`	Taller 2
2 · Último arranque y uptime	`uptime -s`, `uptime -p`	`LastBootUpTime`	Taller 2
3 · Desglose del arranque	`systemd-analyze time`	FwPOSTTime · evento 100	Taller 2
4 · 5 unidades más lentas	`systemd-analyze blame`	Servicios y demoras	Casa
5 · Módulos / controladores	`lsmod`	`driverquery`	Casa
6 · Cierre	Target y servicios fallidos	Inicio rápido	Casa

Entrega

`semana03/informe_arranque.sh` y `.ps1` en el repositorio de la pareja; commits de ambos.

Rúbrica del portafolio

Funciona en ambos SO · documentado · organizado · commits de los dos integrantes.

Parten de `informe_arranque_base.sh/.ps1`.

Fecha: antes de la sesión 1 de la semana 4 (o la que indique el docente).

IA: permitida declarando su uso en la cabecera.

Temas para revisión autónoma

Lo que no entró en clase y conviene leer

SO distribuidos y de red	*MOS §1.4 · Stallings §2.4*
Estructura por capas (THE) y exokernel	*MOS §1.7.2 y §1.7.5*
MBR frente a GPT	*Se retoma en la semana 15*
Secure Boot y la cadena de confianza	*UEFI Specification · documentación de RHEL*
Runlevels de SysV frente a targets de systemd	*bootup(7) · systemd.special(7)*
Hibernar frente a suspender	*Microsoft Learn · systemd-sleep(8)*
Núcleo híbrido en detalle (Windows NT, XNU)	*MOS cap. 11*
Secuencia de arranque de Windows (smss, wininit, winlogon, services)	*Microsoft Learn: Boot process*
initramfs y dracut	*dracut(8) · documentación de RHEL*
SO móviles y embebidos (Android, RTOS)	*MOS §1.4 y cap. 12*

Entran en la prueba parcial de la semana 4 solo en lo que se vio en clase; esta lista es para profundizar.

Conclusiones de la semana

¿Quién es el administrador, qué hace y cómo toma el control cada mañana?



Es el administrador de la cocina: simplifica, reparte y protege; nació para que la CPU no se quede sin trabajo durante la E/S.



Se organiza en departamentos; su oficina puede ser monolítica, microkernel o híbrida.



Toma el control en 5 pasos y lo devuelve con una parada ordenada.

Semana 4

El local está abierto y la ventanilla también: ¿cómo se le hacen pedidos al administrador?

POSIX y Win32 · llamadas al sistema · shell y PowerShell · paquetes · prueba parcial en la sesión 3

Bibliografía y recursos de interés

Bibliografía

- Tanenbaum, A. S. y Bos, H. (2023). Modern Operating Systems (5.ª ed.). Pearson. Caps. 1, 2 y 10.
- Stallings, W. (2018). Operating Systems: Internals and Design Principles (9.ª ed.). Pearson. Cap. 2.
- Carretero, J., García, F., de Miguel, P. y Pérez, F. (2007). Sistemas operativos: una visión aplicada (2.ª ed.). McGraw-Hill. Cap. 2.
- freedesktop.org. systemd-analyze(1), bootup(7), systemd.mount(5).
- Red Hat (2025). RHEL 10: Managing, monitoring and updating the kernel.
- Microsoft Learn. Fast startup; Windows boot process; shutdown.

Recursos

Laboratorio del Administrador

claude.ai/artifact/AXGEhC4p6Ue8wAKNLhDHSf

JSLinux

bellard.org/jslinux · arranca un Linux en el navegador

v86

copy.sh/v86 · emulador x86 en el navegador

explainshell

explainshell.com · explica cada opción de un comando

Mapa del núcleo Linux

makelinux.github.io/kernel/map